

算法原理 Lab2 实验报告

利用动态规划算法求解 TSP 问题

姓名：冯峻 学号：523031910148

2025 年 12 月 6 日

目录

1	实验目的	2
2	算法原理	2
2.1	TSP 问题描述	2
2.2	动态规划解法	2
2.2.1	状态定义	2
2.2.2	状态转移方程	2
2.2.3	算法流程	3
2.3	算法优化策略	3
3	实验内容与结果	5
3.1	实验步骤	5
3.2	实验结果	5
3.2.1	动态规划算法结果	5
3.2.2	子问题统计结果	6
3.2.3	近似算法结果	6
4	结果分析	7
4.1	算法优化效果分析	7
4.2	精确算法与近似算法对比	7
4.3	算法复杂度分析	8
5	结论与实验心得	9

1 实验目的

本实验旨在利用动态规划算法求解旅行商问题 (Traveling Salesman Problem, TSP), 并通过实现和优化多个版本的算法, 深入理解动态规划的基本思想、状态转移方程的设计以及算法优化技巧。

2 算法原理

2.1 TSP 问题描述

旅行商问题 (TSP) 是组合优化领域的经典 NP 难问题。给定 n 个结点和每对结点之间的距离, 要求找出一条访问每个结点恰好一次并返回起点的最短路径。形式化定义为:

给定完全加权图 $G = (V, E)$, 其中 $V = \{0, 1, \dots, n-1\}$ 为结点集合, d_{ij} 表示结点 i 到结点 j 的距离, 目标是找到一个排列 π , 使得:

$$\sum_{i=0}^{n-1} d_{\pi[i], \pi[(i+1) \bmod n]} \quad (1)$$

达到最小。

2.2 动态规划解法

2.2.1 状态定义

定义状态 $dp[S][i]$ 表示从当前结点 i 出发, 访问集合 S 中的所有结点, 最后回到初始结点 0 的最短路径长度。其中:

- S 是一个结点集合, 用二进制掩码表示, 第 k 位为 1 表示结点 k 在集合中, 即结点 k 待访问
- $i \in S$, 且 $0 \in S$ (起点必须在集合中)
- 最终答案为 $\min_{i=1}^{n-1} \{dp[V - \{i\}][i] + d_{0,i}\}$, 其中 V 表示所有结点的集合

2.2.2 状态转移方程

初始状态: $dp[\{0\}][0] = 0$

对于 $|S| \geq 2$ 的递归方程:

$$dp[S][i] = \min_{k \in S, k \neq 0} \{dp[S - \{k\}][k] + d_{i,k}\} \quad (2)$$

该递归方程的含义是: 要计算 $dp[S][i]$, 需要考虑从集合 S 中除 i 外的每个结点 k 出发, 从 i 到 k , 然后从 k 访问完 $S - \{i\}$ 中的所有结点的总路径, 取其中的最小值。

2.2.3 算法流程

Algorithm 动态规划求解 TSP

输入: 邻接矩阵 $adj[n][n]$, 结点数量 n

输出: 最短路径长度和路径序列

- 1: 初始化 $dp[\{0\}][0] = 0$, 其余为 ∞
 - 2: **for** $s = 2$ to n **do** ▷ 按照集合大小递增, 避免变量名冲突
 - 3: **for** 所有大小为 s 且包含结点 0 的集合 S **do**
 - 4: **for** $i \in S$ 且 $i \neq 0$ **do**
 - 5: $dp[S][i] = \infty$
 - 6: **for** $j \in S$ 且 $j \neq i$ **do** ▷ 枚举前驱结点 j
 - 7: $dp[S][i] = \min(dp[S][i], dp[S \setminus \{i\}][j] + adj[j][i])$
 - 8: $result = \min_{i=1}^{n-1} \{dp[V][i] + adj[i][0]\}$ ▷ 加上回到起点的距离
 - 9: **return** $result$ 和通过记录路径回溯得到的完整路径
-

2.3 算法优化策略

本实验实现了四个版本的动态规划算法, 每个版本在前一版本基础上进行了优化:

1. **solve_TSP_with_dp:** 基础版本, 使用哈希表 (`unordered_map`) 存储状态
这样做的好处就是每次只会使用到两层状态: 当前集合大小 k 和 $k-1$ 的状态, 因此可以通过迭代数组技术节省空间。
2. **solve_TSP_with_dp_fast:** 使用二维数组 (`vector<vector>`) 替代哈希表, 提高访问效率
经过测试之后发现哈希表结点权重太高, 所以想尝试直接使用二维数组存储状态。但是发现如果用二维数组存储, 所有状态都会被初始化, 导致内存开销过大。测试之后虽然发现在 att7 和 att20 数据集上性能提升了 3-4 倍, 但在更大数据集上会因为内存不足而无法运行。因此接下来尝试进一步优化状态空间。
3. **solve_TSP_with_dp_fast_v2:** 利用起点固定为 0 的性质, 将状态空间减半
由于我们考虑的所有掩码序列 `mask_index`, 其 LSP 都被强制规定为 1, 因此我们就没有必要存储那些 LSP 为 0 的掩码序列对应的状态。这样一来, 状态空间就从 $O(2^n \cdot n)$ 减少到 $O(2^{n-1} \cdot n)$, 节省了一半的内存开销。经过测试, 在 att7 和 att20 数据集上分别提升了约 18% 和 6.6% 的性能。
4. **solve_TSP_with_dp_fast_v3:** 迭代二维数组, 进一步优化空间复杂度

算法 fast_v3 综合考虑了算法 dp 和 dp_fast 的优点：(1) 使用二维数组 (vector<vector>) 存储状态，提升访问效率；(2) 每次循环只记录当前集合大小 k 和 $k-1$ 的状态，节省空间开销。具体做法如下：

我们在每次循环时，只需要拥有结点个数为 $k-1$ 的掩码，然后根据该信息生成节点个数为 k 的掩码集合。但是这些掩码的值是离散的，无法直接使用 vector 存储，因此，我们需要采用一种方法，将这些离散的掩码 (含有 k 个 1 的二进制序列) 映射到连续的索引上。这种映射关系，需要我们实现：假设对于 N 位宽的二进制序列，我们记 1 的个数为 N 的二进制序列集合为 S_N ，对于任意一个含有 N 个 1 的二进制序列 a ，我们需要一个函数 $f(a)$ ，能够在不直接计算 S_N 中的所有序列的情况下，返回 a 在升序排列后的 S_N 中的索引位置。

经过查找，我们确实找到了这种方法，如下：

组合数编号法 设有一个宽度为 M 的二进制掩码 a ，其中恰有 N 个 1。记 S_N 为所有“宽度为 M 、恰有 N 个 1”的掩码集合，并按数值从小到大排序。我们的目标是在不显式生成 S_N 的情况下，直接计算 a 在 S_N 中的索引 (从 0 开始)。

令 a 中从低位到高位 1 的位置为 $0 \leq p_1 < p_2 < \dots < p_N \leq M-1$ ，则 a 的索引由下式给出：

$$\text{index}(a) = \sum_{i=1}^N \binom{p_i}{i}. \quad (3)$$

示例： $M=5, N=3, a=10110_2$ ，其 1 的位置为 $[1, 2, 4]$ ，则

$$\text{index}(a) = \binom{1}{1} + \binom{2}{2} + \binom{4}{3} = 1 + 1 + 4 = 6.$$

该方法等价于将“从 M 位中选 N 位为 1”的组合做秩 (rank) 编号，时间复杂度 $O(M)$ ，无需枚举全部组合，适用于将“固定 1 的个数”的子集映射到连续内存索引，从而实现 DP 层内的紧凑存储。

理论上，空间复杂度可以降低到 $O(C_n^k \cdot n)$ ，其中 k 是当前处理的集合大小。但实验结果显示，在 att7 和 att20 数据集上性能反而下降了。这是因为 v3 需要频繁计算组合数索引，额外的计算开销超过了节省内存带来的收益。

3 实验内容与结果

3.1 实验步骤

1. 实现基础动态规划算法 `solve_TSP_with_dp`，后续不断更新迭代另外三个算法
2. 为每个算法添加子问题统计功能，记录：
 - 所有子问题访问次数（包括重复访问）
 - 首次出现的子问题数量（不计重复）
3. 实现 2-opt 近似算法作为对照（**2-opt 部分算法为 AI 生成**）
4. 在不同数据集上测试各算法的性能
5. 对比分析动态规划精确算法与近似算法的差异

3.2 实验结果

3.2.1 动态规划算法结果

att7 数据集结果

算法版本	运行时间 (ms)	最短路径长度	相对速度
<code>solve_TSP_with_dp</code>	0.1778	126	1.00×
<code>solve_TSP_with_dp_fast</code>	0.0485	126	3.67×
<code>solve_TSP_with_dp_fast_v2</code>	0.0413	126	4.31×
<code>solve_TSP_with_dp_fast_v3</code>	0.0772	126	2.30×

表 1: att7 数据集动态规划算法性能对比

最优路径: $0 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 0$

att20 数据集结果

算法版本	运行时间 (ms)	最短路径长度	相对速度
<code>solve_TSP_with_dp</code>	4908.61	7260	1.00×
<code>solve_TSP_with_dp_fast</code>	1309.60	7260	3.75×
<code>solve_TSP_with_dp_fast_v2</code>	1223.09	7260	4.01×
<code>solve_TSP_with_dp_fast_v3</code>	5402.87	7260	0.91×

表 2: att20 数据集动态规划算法性能对比

最优路径: $0 \rightarrow 7 \rightarrow 8 \rightarrow 14 \rightarrow 17 \rightarrow 6 \rightarrow 5 \rightarrow 18 \rightarrow 16 \rightarrow 19 \rightarrow 11 \rightarrow 10 \rightarrow 12 \rightarrow 13 \rightarrow 4 \rightarrow 9 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 15 \rightarrow 0$

bays29 数据集结果

很遗憾，我实现的所有版本的动态规划算法在 bays29 数据集上无法在一个小时之内完成计算，内存消耗过大，因此未能获得结果。

3.2.2 子问题统计结果

通过为算法添加统计功能，我们获得了动态规划过程中的子问题访问情况：

数据集	总子问题访问次数	唯一子问题数量
att7	492	193
att20	44,826,662	4,980,737

表 3: 动态规划子问题统计

从统计结果可以看出：

- 平均每个唯一子问题被访问约 $\frac{492}{193} \approx 2.55$ 次（att7）
- 平均每个唯一子问题被访问约 $\frac{44826662}{4980737} \approx 9.00$ 次（att20）
- 随着结点数量增加，子问题的重复访问率显著提高，这凸显了动态规划记忆化的重要性

3.2.3 近似算法结果

使用 2-opt 近似算法在三个数据集上的结果：

数据集	运行时间 (ms)	路径长度	最优值	近似比
att7	0.0115	149	126	1.18
att20	0.0358	8185	7260	1.13
bays29	0.0702	2064	-	-

表 4: 2-opt 近似算法结果

4 结果分析

4.1 算法优化效果分析

1. 哈希表 vs 数组 (dp vs dp_fast)

使用二维数组替代哈希表后, 在 att7 上速度提升了 3.67 倍, 在 att20 上提升了 3.75 倍。这是因为:

- 数组访问具有 $O(1)$ 的常数时间复杂度, 而哈希表需要计算哈希值并处理冲突
- 数组访问具有更好的缓存局部性

2. 状态空间压缩 (dp_fast vs dp_fast_v2)

利用起点固定的特性将状态空间减半后, 在 att7 上进一步提升了约 18% 的性能 (从 0.0485ms 降至 0.0413ms)。在 att20 上也有约 6.6% 的提升。空间复杂度从 $O(2^n \cdot n)$ 降至 $O(2^{n-1} \cdot n)$ 。

3. 迭代更新数组 (dp_fast_v2 vs dp_fast_v3)

v3 版本迭代更新数组, 这样不必记录所有的掩码状态, 理论上可以将空间复杂度降低到 $O(C_n^k \cdot n)$, 其中 k 是当前处理的集合大小。但实验结果显示:

- 在 att7 上, v3 反而比 v2 慢 (0.0772ms vs 0.0413ms)
- 在 att20 上, v3 比 v2 慢得多 (5402.87ms vs 1223.09ms)

这是因为 v3 需要频繁计算组合数索引, 额外的计算开销超过了节省内存带来的收益。在内存充足的情况下, v2 是最优选择。

4.2 精确算法与近似算法对比

数据集	动态规划 (最快版本)		2-opt 近似算法	
	时间 (ms)	路径长度	时间 (ms)	路径长度
att7	0.0413	126	0.0115	149
att20	1223.09	7260	0.0358	8185

表 5: 精确算法与近似算法对比

分析:

- **解的质量：**动态规划能够找到最优解，而 2-opt 只能找到局部最优解。对于 att7 和 att20，近似算法的解分别比最优解差 18% 和 12.7%。
- **运行时间：**近似算法的速度优势明显。对于 att20，近似算法比最优的动态规划算法快约 34000 倍 ($1223.09/0.0358 \approx 34167$)。
- **可扩展性：**动态规划的时间复杂度为 $O(n^2 \cdot 2^n)$ ，空间复杂度为 $O(n \cdot 2^n)$ ，在 $n=29$ 时已经无法在合理时间内完成。而近似算法可以轻松处理 bays29 数据集。

4.3 算法复杂度分析

算法	时间复杂度	空间复杂度	适用规模
DP 基础版	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$	$n \leq 20$
DP 优化版 v3	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^{n-1})$	$n \leq 20$
2-opt 近似	$O(n^3)$	$O(n^2)$	$n \leq 10000$

表 6: 算法复杂度对比

5 结论与实验心得

通过本次实验，我们深入理解了动态规划算法在求解 TSP 问题中的应用，得出以下结论：

1. **动态规划有效性：**基于状态压缩的动态规划算法能够在 $O(n^2 \cdot 2^n)$ 时间内求解 TSP 问题的最优解，适用于 $n \leq 20$ 的中小规模问题。一旦规模过大，指数级复杂度在有限时间内无法求解。
2. **优化策略的重要性：**通过使用数组代替哈希表、压缩状态空间等优化手段，可以将算法性能提升 3-4 倍。但并非所有理论上的优化都能带来实际收益（例如 v3 利用组合数索引方法本意是想降低存储负担，但是带来了大量的计算过程，增大了时间复杂度），需要权衡计算开销和内存访问效率。
3. **子问题重复访问：**统计数据表明，动态规划过程中存在大量的子问题重复访问（平均 9 次/att20），验证了记忆化搜索的必要性。

通过本次实验，我对动态规划算法有了更深入的理解：

- 掌握了状态压缩、迭代更新数组等常用的动态规划优化技巧
- 体会到了精确算法和近似算法各自的优势与局限性
- 提高了 C++ 编程能力和性能分析能力

本实验加深了我对算法设计与分析的理解，为今后解决更复杂的优化问题奠定了良好的基础。